

Fuzzy Q-Learning with Fuzzified Bellman Equation for Unmanned Ground Vehicle Navigation

Mohsen Jalaiean Farimani*, Davoud Nikkhouy†, Mahshad Rastegarmoghaddam‡,
Shima Samadzadeh, Saeed Mozaffari†, Shahpour Alirezaee†,

*Departement of Electronic, Information and Bioengineering (DEIB), Politecnico di Milano, Italy

†Mechanical, Automotive and Materials Engineering (MAME), University of Windsor, Canada

‡Department of Mechanical Engineering, Politecnico di Milano, Italy

*Corresponding author: mohsen.jalaiean@polimi.it,

davoudnikkhouy@gmail.com, mahshad.rastegarmoghaddam@mail.polimi.it, shima.samadzade@gmail.com,

saeed.mozaffari@uwindsor.ca, s.alirezaee@uwindsor.ca.

Abstract—This paper presents a novel Fuzzy Q-Learning with Fuzzified Bellman Equation (FQL-FBE) that is utilized to advance autonomous navigation in dynamic and unknown environments. A modified Fuzzified Bellman equation is introduced to integrate fuzzy logic into reinforcement learning by updating the Q-table using Fuzzy temporal differences. The proposed approach serves as a state-action value estimator, addressing challenges associated with continuous state-action spaces by introducing discretization for enhanced learning efficiency. Key advantages include low computational complexity, high adaptability to changing environmental conditions, effective handling of complex scenarios, smooth path generation, interpretability, and a fast learning process. The results of simulations and comparisons validate the efficiency of the proposed method, demonstrating enhanced obstacle avoidance, improved path efficiency, and reduced computational cost. This work marks a step forward in the development of robust and efficient solutions for real-time robotic navigation.

Index Terms—Fuzzy Q-Learning, Reinforcement Learning, Navigation, Path Planning, ROS2, Gazebo, Turtlebot3,

I. INTRODUCTION

Navigation is a fundamental problem in robotics and autonomous systems, involving the determination of an optimal path while avoiding obstacles and adhering to task-specific constraints [1]. Applications range from self-driving cars and robotic manipulators to drones and unmanned surface vehicles, making it a critical domain of study. However, navigating dynamic, uncertain environments with continuous state and action spaces remains a significant challenge. These environments introduce infinite possibilities for decision-making, which demand advanced algorithms capable of real-time adaptation and robust learning.

Traditional methods, such as graph-based algorithms like A* [2], have limitations in real-world scenarios due to their computational inefficiency and inability to handle dynamic obstacles. Similarly, potential field methods, while simple and intuitive, often suffer from local minima and instability issues [3] and [4]. To overcome these drawbacks, reinforcement learning (RL) [5], has emerged as a powerful approach that enables systems to learn optimal behaviors through interactions with the environment [6]. Despite its promise, RL-based strategies, such as deep

Q-learning and policy gradient methods, often face challenges related to high computational complexity, convergence in large state-action spaces, and real-time adaptability [7].

Fuzzy logic, with its ability to partition input spaces into interpretable subsets, has been explored as a complementary tool for RL. Early works, such as Berenji's Fuzzy RL [8], demonstrated its potential but were hindered by computational challenges and limited interpretability. Recent advancements have combined Fuzzy logic with RL to enhance decision-making in complex environments, but existing solutions often fail to balance efficiency, scalability, and robustness [9].

This research introduces a Novel Fuzzy Q-Learning with Fuzzified Bellman Equation (FQL-FBE) framework that bridges this gap. By leveraging Fuzzy partitioning to discretize continuous state and action spaces, the proposed method integrates the adaptability of RL with the interpretability and efficiency of Fuzzy logic. This combination results in a navigation system capable of dynamic decision-making, smooth trajectory generation, and low computational overhead.

The main contributions of this paper, which distinguish it from existing methods, can be stated as follows.

- (1) A novel FQL-FBE framework that utilizes Fuzzy membership functions for efficient state and action space discretization, enabling tabular Q-learning in continuous domains.
- (2) A modified Fuzzified Bellman equation is proposed that seamlessly integrates Fuzzy logic into reinforcement learning for robust update of the Q-value.
- (3) A modified action selection to bring the concept of Fuzziness into tabular Q-Learning greedy action selection approach.
- (4) High-dimensional sensor data, such as Lidar, are pre-processed by dimensionality reduction and Fuzzification to improve interpretability and decision making.
- (5) Compared with similar recent methods, the method shows superior performance in dynamic and unknown environments, achieving smoother paths, efficient obstacle avoidance, and low computational cost, as validated through simulations with TurtleBot3 in ROS2-Gazebo.

The proposed Fuzzy Q-Learning with Fuzzified Bellman Equation (FQL-FBE) fundamentally extends traditional Q-

learning, Eq. (2), by integrating fuzzy logic into three key components: 1- state-action space partitioning via Gaussian membership functions, Eqs. (3) and (4), which enables continuous interpolation; 2- a fuzzified TD update, Eqs. (7)-(10), where the Bellman equation is weighted by membership values ($\zeta_{i,j}$) and future rewards are estimated through fuzzy aggregation ($\Upsilon(s')$); and 3- policy execution that defuzzifies actions via weighted inference, Eq. (5), rather than discrete maximization. This hybrid approach preserves the interpretability of tabular Q-learning while addressing continuous state-action challenges through fuzzy rule-based generalization. The framework's robustness stems from its inherent capacity to handle sensor noise and environmental uncertainty through graded membership activations, bridging the precision of reinforcement learning with the adaptability of fuzzy systems.

II. Q-LEARNING'S FUNDAMENTALS

In this section, the proposed Fuzzy Q-Learning algorithm is described in detail. First, the main fundamental concepts that are used are briefly described:

1) *State-Action Value (Q)*: The state-action value Q indicates the expected cumulative reward (Return) in the long term resulting from taking action a in state s under policy π . This concept is mathematically defined in (1), where $\mathbb{E}$ represents the mathematical expectation.

$$Q^\pi(s, a) = \mathbb{E}[r_{t+1} + \lambda Q^\pi(s', a') \mid s, a] \quad (1)$$

2) *Policy (π)*: The policy is responsible for generating the proper action in each state. This means that the (*behavior policy*) generates control actions within the environment, and at the same time, the learning process trains the *target policy* through an optimization method.

3) *Policy improvement*: Based on Q-Learning Algorithm, the action-value function ($Q(s, a)$) is updated according to the Bellman equation (2), where $0 < \alpha < 1$ is the learning rate, and $0 < \gamma \leq 1$ is the discount factor. The maximum state action value for the next state is defined as $Q'_{max} = \max_{a'} Q(s', a')$

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q'_{max} - Q(s, a)) \quad (2)$$

The Q-learning algorithm is a model-free RL algorithm. It is based on policy optimization and seeks to maximize the action-value function ($Q(s, a)$). Due to its ability to learn directly from experiences, Q-Learning algorithm considers specifying areas of application (e.g., robotic navigation and control). In discrete state and action spaces, Q-Learning can be simply implemented in a tabular manner. However, in continuous state or action spaces, the tabular implementation is not applicable. Therefore, a non-linear function as an approximator must be utilized, which brings a layer of extra complexity into the learning process.

III. PROPOSED FUZZY Q-LEARNING METHOD

Here, Fuzzy Q-Learning with Fuzzified Bellman Equation (FQL-FBE) is presented as an alternative approach, to estimate the state-action value function, $Q(s, a)$. In the proposed FQL-FBE, the state and action spaces will be Fuzzified through two sets of Fuzzy membership functions.

1) *State and Action Spaces Fuzzification*: To discretize both the state and action spaces, two complete and consistent sets of Gaussian membership functions ($\mu_S(s) \in R^{N_s}$ and $\mu_A(a) \in R^{N_a}$) are defined, which are described in (3) and (4). The predefined parameters of these membership functions are their centers (c_s and c_a) and standard deviations of them (σ_s and σ_a). Their centers uniformly distributed across the respective input domains. The standard deviations are selected to ensure sufficient overlap between adjacent functions, promoting smooth transitions. The number of membership functions, N_s for states and N_a for actions, is empirically set based on preliminary sensitivity analyses to balance granularity and computational load. The rule base is constructed implicitly through the Q-table, where each entry corresponds to a specific state-action membership pair, and its updates are governed by the fuzzified Bellman equation. This systematic configuration ensures consistency and facilitates replication of the FQL-FBE framework. For better understanding see Fig. 1.

$$\mu_S(s) = \exp\left(-\frac{(s - c_s)^2}{2\sigma_s^2}\right) \quad (3)$$

$$\mu_A(a) = \exp\left(-\frac{(a - c_a)^2}{2\sigma_a^2}\right) \quad (4)$$

2) *Q-Table*: A numerical table ($\hat{Q} \in R^{N_s \times N_a}$) is initialized with random values such that after the train process, it represents the estimation of $Q(s, a)$ for the state-action pairs.

3) *FQL-FBE process*: The training process of the proposed approach is explained in the following.

- *Current state (s)*: The agent determines the current state.
- *State Fuzzification ($\mu_S(s)$)*: the membership values of the current state ($\mu_S(s)$) are computed.
- *Finding $a_i = \arg\max_a (\hat{Q}(s_i, a)) \forall s_i$* : According to the estimated $\hat{Q}$ table, for all state membership functions, find the action membership function that provides the maximum value.

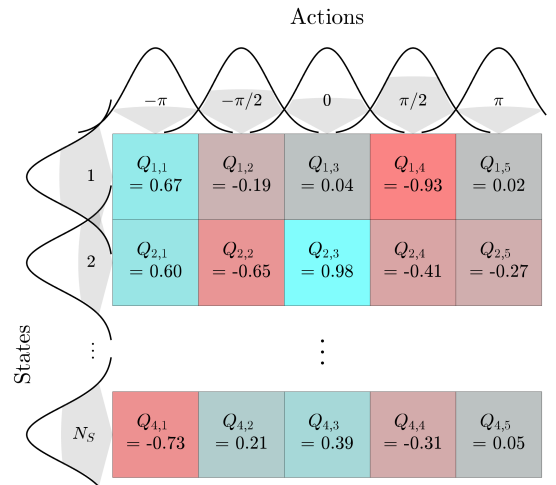


Fig. 1: Lidar data integration into navigation commands.

- *Greedy Action Selection (a^*):*

$$a^* = \frac{\sum_{i=1}^{N_s} (\mu_S(s_i) \hat{Q}(s_i, a_i) a_i)}{\sum_{i=1}^{N_s} (\mu_S(s_i) \hat{Q}(s_i, a_i))} \quad (5)$$

- ϵ -Greedy Action Selection (a): If a randomly generated number is less than a predefined parameter (ϵ), then a random action is applied by the agent to perform in the current state. Otherwise, the greedy action (a^*) will be applied.

$$a = \begin{cases} a^*, & \text{with probability } 1 - \epsilon, \\ \text{a random action}, & \text{with probability } \epsilon. \end{cases} \quad (6)$$

- *Reward (r):* A reward is assigned for the chosen action.
- *Transition to a new state (s'):* The agent transitions to a new state.
- *Update $\hat{Q}$ table:* The estimated action-value table is updated according to the proposed Fuzzified Bellman Equation (7), where $0 < \alpha < 1$ is the learning rate, and $0 < \gamma \leq 1$ is the discount factor. $\forall i \in [1, \dots, N_s], \forall j \in [1, \dots, N_a]$:

$$\hat{Q}_{i,j} \leftarrow \hat{Q}_{i,j} + \alpha \zeta_{i,j} \varpi_{i,j}(s') \quad (7)$$

where, $\zeta_{i,j}$ is the element of i^{th} row and j^{th} column of $\zeta(s, a) \in R^{N_s \times N_a}$ that represent the value in which each element of the $\hat{Q}$ table was involved in the selected action.

$$\zeta(s, a) = \mu_S(s) \mu_A(a)^\top \quad (8)$$

and $\varpi_{i,j}(s')$ is the Fuzzy temporal difference, as formulated below.

$$\varpi_{i,j}(s') = \left(r + \gamma \Upsilon(s') - \hat{Q}_{i,j} \right) \quad (9)$$

in which, $\Upsilon(s')$ is the Fuzzified maximum value possibly offered by the next state.

$$\Upsilon(s') = \sum_{i=1}^{N_s} \left(\frac{\mu_{Si}(s')}{\sum_{i=1}^{N_s} \mu_{Si}(s')} \max_j \hat{Q}_{i,j} \right) \quad (10)$$

- *Repeat:* The procedure is repeated, and training continues until the policy converges.

This method enables the robot to interact continuously with the environment, improving its decision-making capabilities, and achieving the intended goal. The process of the proposed FQL-FBE is described in Algorithm 1.

IV. NAVIGATION PROBLEM

The navigation process relies on sensory inputs to perceive the environment and issue appropriate control commands. Lidar (Light Detection and Ranging) sensors play a pivotal role in generating accurate 2D/3D maps of the surroundings, detecting obstacles, and measuring distances. These sensor outputs are processed into control commands, such as linear and angular velocities, which are transmitted to the robot's actuators to execute navigation tasks. The integration of sensor data into control systems ensures precise and adaptive responses.

Algorithm 1 The Proposed FQL-FBE

- 1: Define the Fuzzy membership functions for state and action spaces, $\mu_S(s)$ and $\mu_A(a)$ and their numbers N_s and N_a .
 - 2: Initialize the $\hat{Q}$ table.
 - 3: Set learning rate α and discount factor γ .
 - 4: **for** each episode **do**
 - 5: Initialize the state s and select action a by the policy.
 - 6: **repeat**
 - 7: Select action a based on (6) and apply it.
 - 8: Receive reward r and next state s' ,
 - 9: **for** $i \in [1, \dots, N_s]$, and $j \in [1, \dots, N_a]$: **do**
 - 10: Compute Fuzzified Maximum Value (10),
 - 11: Compute the Fuzzy Temporal Difference (9),
 - 12: Compute the involving value (8),
 - 13: Update $\hat{Q}$ table (7),
 - 14: **end for**
 - 15: $s \leftarrow s'$,
 - 16: **until** s' is a terminal state or the maximum number of steps is exceeded
 - 17: **end for**
-

A. Problem Statement

Robotic navigation in dynamic and uncertain environments presents significant challenges due to real-time decision making, balancing local and global objectives, and environmental uncertainties and nonlinear dynamics. These challenges necessitate advanced navigation algorithms capable of real-time adaptation, efficient resource utilization, and robust performance in uncertain and dynamic scenarios. The proposed FQL-FBE is an advanced technique for navigation, which combines reinforcement learning with Fuzzy logic to map sensory inputs to control signals. The method employs a Q-function, optimized through experience, to determine the best control actions based on the current state. The NFQL framework involves:

- **State Representation:** The robot's state s_t is defined by sensor readings (e.g., Lidar) and the target's location and robot's position and orientation.
- **Action Space:** Control signals, such as velocity commands $a_t = [v, \omega]$, define the action space.
- **Reward Function:** Reward function $R(s_t, a_t)$ is formulated to encourage goal-reaching while penalizing collisions or inefficiency as follows.

$$R = w_1 f_{\text{path}}(t) + w_2 f_{\text{obstacle}}(t) + w_3 f_{\text{energy}}(t) + w_4 f_{\text{goal}}(t) \quad (11)$$

where w_1 , w_2 , w_3 , and w_4 represent the weights for path efficiency, obstacle avoidance, energy consumption, and alignment with the goal direction, respectively. The functions f_{goal} can be designed to measure alignment with the goal's direction and the proximity to the goal, ensuring effective and efficient navigation. Established methods like Artificial Potential Fields (APF) or Model Predictive Control (MPC) can be adapted to compute these $f(t)$ terms [10].

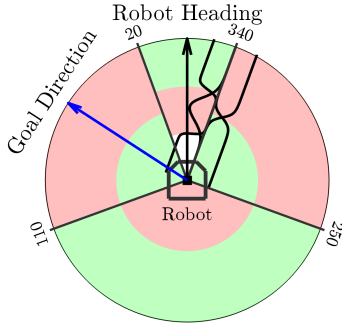


Fig. 2: Fuzzification of Lidar Inputs for State Representation

B. Proposed Navigation Approach

The proposed approach integrates FQL with real-time sensor data to address the navigation problem effectively. The methodology consists of:

- 1) Pre-processing of Lidar data, which includes reducing the input dimensionality and Fuzzifying the sensor readings to enhance interpretability.
- 2) Utilizing Q-Learning to enable adaptive decision-making and robust navigation.
- 3) Fuzzifying sensor inputs and Defuzzifying control actions to bridge tabular Q-Learning algorithms and Fuzzy systems.

The preprocessing of Lidar data is a critical step to manage the high-dimensional raw input effectively. By reducing the dimensionality and applying Fuzzification techniques, the sensor readings are transformed into a more compact and interpretable state representation. This step is visually represented in Fig. 2, which illustrates the process of converting raw Lidar data into Fuzzy states for use in decision-making.

The integration of Fuzzy logic with Q-Learning forms the foundation of the proposed navigation system. This hybrid approach leverages the adaptive learning capabilities of Q-Learning to optimize decision-making while utilizing Fuzzy logic to handle the inherent uncertainties in sensor inputs and environment dynamics. The use of Fuzzification ensures that the system can interpret sensor data effectively, and Defuzzification translates the learned actions into precise control signals. This methodology enables the robot to navigate efficiently, avoiding obstacles and reaching its target in dynamic environments.

The implementation and simulation of the proposed methodology are depicted in Fig. 3. The block diagram outlines the system architecture, showcasing the flow of data from sensor preprocessing, through the Fuzzy Q-Learning algorithm, to the execution of control actions. This comprehensive diagram highlights the modularity and scalability of the proposed system, making it well-suited for real-world robotic applications.

V. SIMULATION

A. Robot and Environment

The proposed FQL-FBE framework was implemented and tested in a ROS2-Gazebo simulation environment [11] us-

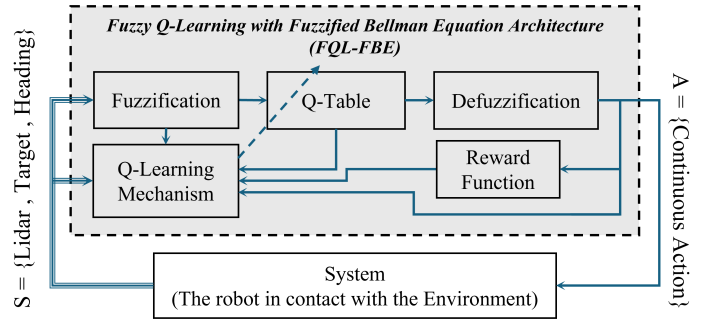


Fig. 3: Block Diagram of the proposed FQL-FBE System

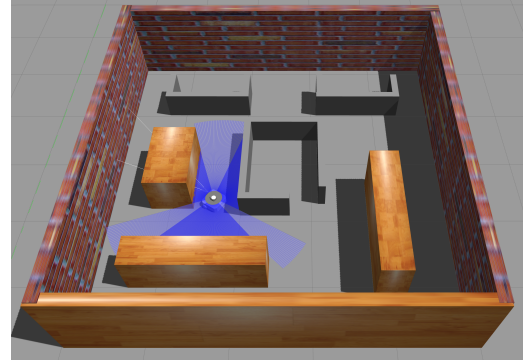


Fig. 4: Simulated Robot and Environment Setup

ing a TurtleBot3 Waffle robot equipped with Lidar sensors. Gazebo provided a realistic simulation of robot dynamics and environmental interactions, while MATLAB handled algorithm design and parameter optimization. Communication between ROS2 and MATLAB ensured real-time data exchange and synchronized operation. The simulated robot and environment setup are depicted in Fig. 4.

B. Implementation and Results

The FQL-FBE algorithm effectively processed Lidar data for state-action mapping and navigation optimization. Fig. 5 illustrates the robot's final navigated path, showcasing smooth and efficient obstacle avoidance. The training process, as shown in Fig. 6, demonstrated stable return convergence over time. The simulation validated the FQL-FBE framework's adaptability and effectiveness in dynamic environments, achieving smooth trajectories, reliable obstacle avoidance, and efficient navigation.

C. Parameter Analysis

The effects of key parameters were analyzed to determine their impact on navigation performance and their respective influences on the efficiency and robustness of the system. After some trial and error, the results showed that for this case study, the best learning rate is $\alpha = 0.01$ and the best discount factor is $\gamma = 0.9$.

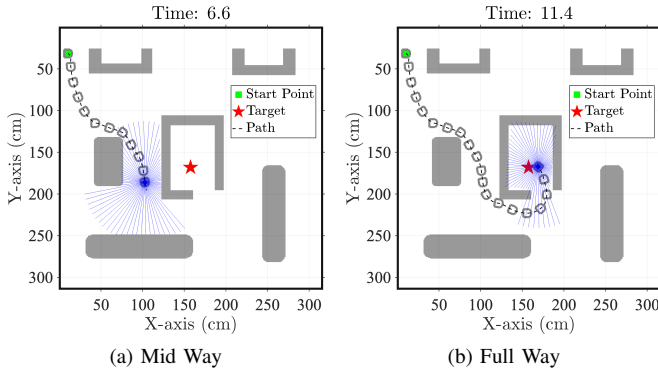


Fig. 5: Robot's Final Navigated Path in the Environment

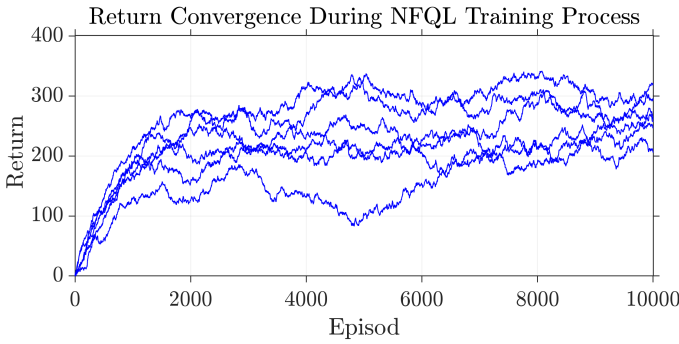


Fig. 6: FQL-FBE Training Process: Return Convergence

D. Comparison Analysis

To demonstrate its superiority, the proposed method is compared against state-of-the-art approaches, including BBA-PER-DDQN, baseline DQN [7], and DWA-VO [9]. The numerical performance is assessed using four key evaluation indices [7], [9]: Success Rate (ScR), Collision Rate (CIR), Coverage Rate (CvR), and Training Time (TrT), with the results summarized in Table I. These indices comprehensively capture critical aspects of robotic navigation, including task completion, safety, exploration efficiency, and computational cost.

TABLE I: Comparative Analysis of Navigation Methods

Methods	ScR (%)	CIR (%)	CvR (%)	TrT (min)
DQN	44	33	75	75
PER-DQN	58	15	68	63
BBA-PER-DDQN	91	8	98	36
DWA-VO	75	22	62	70
FQL-FBE*	93	4	88	28

*Proposed method.

The proposed FQL-FBE achieves the highest success rate (93 %), the lowest collision rate (4%), and the fastest training time (28 minutes), demonstrating superior adaptability and safety in dynamic environments. While BBA-PER-DDQN excels in static exploration tasks (98% coverage), FQL-FBE prioritizes real-time navigation, reducing collision risk by 50% compared to BBA-PER-DDQN and training 22% faster. The DWA-VO

method, designed for dynamic collision avoidance, achieves a 75% success rate but incurs higher computational costs (70 minutes). These results highlight the advantages of integrating fuzzy logic with reinforcement learning, enabling robust performance in both exploration and dynamic obstacle avoidance tasks.

VI. CONCLUSION

In this study, we introduced a novel Fuzzy Q-Learning with Fuzzified Bellman Equation (FQL-FBE) framework, tailored for robotic navigation in dynamic and unknown environments. The conventional Bellman equation is modified in this paper into a Fuzzified version. The integration of fuzzy logic with reinforcement learning enabled the effective partitioning of continuous state and action spaces, resulting in improved adaptability, computational efficiency, and smoother navigation trajectories. The simulation results using TurtleBot3 and the ROS2-Gazebo framework demonstrated the effectiveness of the proposed method, compared to other similar recent methods, in terms of obstacle avoidance and path efficiency. The proposed FQL-FBE algorithm provides a robust and interpretable solution for real-time navigation in complex environments. Future work could explore extending this framework to multi-agent scenarios and implementing the method in real-world environments for validation.

REFERENCES

- [1] D. Damodaran, S. Mozaffari, S. Alirezadeh, and M. J. Ahamed, "Experimental analysis of the behavior of mirror-like objects in lidar-based robot navigation," *Applied Sciences*, vol. 13, no. 5, 2023.
- [2] D. Xu, J. Yang, X. Zhou, and H. Xu, "Hybrid path planning method for usv using bidirectional a* and improved dwa considering the manoeuvrability and colregs," *Ocean Engineering*, vol. 298, p. 117210, 2024.
- [3] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang, "Path planning techniques for mobile robots: Review and prospect," *Expert Systems with Applications*, vol. 227, p. 120254, 2023.
- [4] S. D. N. Tanha, S. F. Dehkordi, and A. H. Korayem, "Control a mobile robot in social environments by considering human as a moving obstacle," in *2018 6th RSI International Conference on Robotics and Mechatronics (ICRoM)*, 2018, pp. 256–260.
- [5] R. K. Amirabadi, O. S. Fard, and M. Jalaeeian-F., "Towards optimal control of hpv model using safe reinforcement learning with actor-critic neural networks," *Expert Systems with Applications*, vol. 264, p. 125783, 2025.
- [6] J. R. Sánchez-Ibáñez, C. J. Pérez-del Pulgar, and A. García-Cerezo, "Path planning for autonomous mobile robots: A review," *Sensors*, vol. 21, no. 23, 2021.
- [7] Y. Zhou, J. Yang, Z. Guo, Y. Shen, K. Yu, and J. Chun-Wei Lin, "An indoor blind area-oriented autonomous robotic path planning approach using deep reinforcement learning," *Expert Systems with Applications*, vol. 254, p. 124277, 2024.
- [8] H. R. Berenji, "A reinforcement learning—based architecture for fuzzy logic control," *International Journal of Approximate Reasoning*, vol. 6, no. 2, pp. 267–292, 1992.
- [9] X. Yang, M. Lou, J. Hu, H. Ye, Z. Zhu, H. Shen, Z. Xiang, and B. Zhang, "A human-like collision avoidance method for usvs based on deep reinforcement learning and velocity obstacle," *Expert Systems with Applications*, vol. 254, p. 124388, 2024.
- [10] H. Ke, S. Mozaffari, S. Alirezadeh, and M. Saif, "Cooperative adaptive cruise control using vehicle-to-vehicle communication and deep learning," in *2022 IEEE Intelligent Vehicles Symposium (IV)*, 2022, pp. 435–440.
- [11] V. Seidita and A. Chella, "Agents as a design paradigm for robotic systems leveraging ros and gazebo," in *Agents and Robots for reliable Engineered Autonomy*, A. Ferrando and R. C. Cardoso, Eds. Cham: Springer Nature Switzerland, 2025, pp. 21–37.